

FRAMEWORK UNIFICADO

Apache Spark

Processamento de Dados em Larga Escala

ETL

Machine Learning

IA

Streaming

Professor Benir Falcão Pistille

Sumário

01

Fundamentos e Arquitetura

Conceitos fundamentais, componentes principais e modelo de execução distribuído

02

Processamento de Dados (ETL)

Pipelines ETL, DataFrames e otimização de processamento em larga escala

03

Spark SQL e DataFrames

Processamento estruturado, Catalyst Optimizer e Tungsten Engine

04

Streaming e Processamento em Tempo Real

Structured Streaming, watermarks e modo em tempo real

05

Machine Learning com MLlib

Algoritmos de ML, pipelines e Data Mining em escala

06

Tecnologias Emergentes

Delta Lake, Spark 4.0, AQE e ecossistema moderno

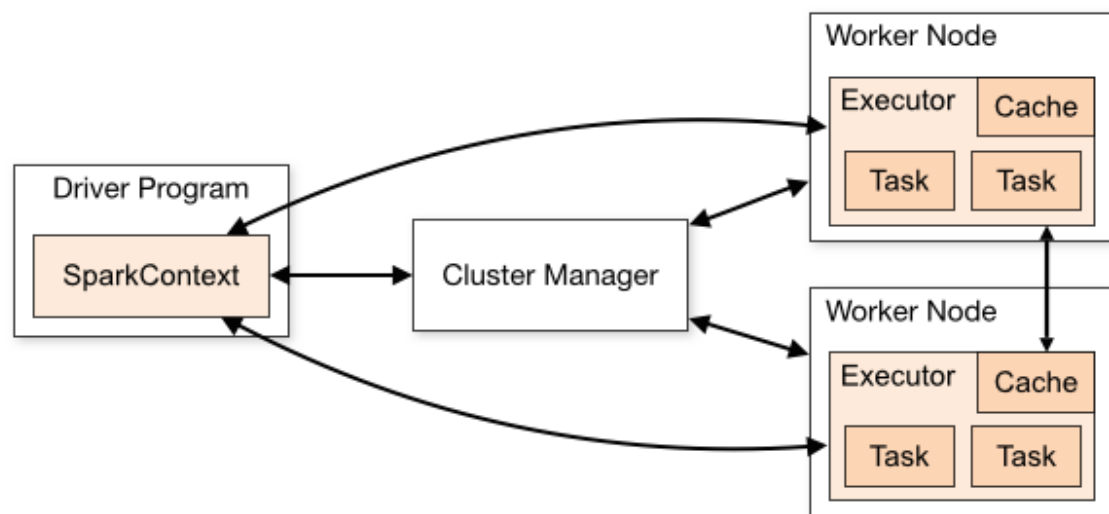
ETAPA 01

Fundamentos e Arquitetura

Conceitos fundamentais da arquitetura Apache Spark, incluindo componentes principais, modelo de execução e estrutura de processamento distribuído



Arquitetura Geral do Apache Spark



Arquitetura Master-Worker: O Spark opera em uma arquitetura distribuída onde o Driver Program coordena a execução de tarefas através de um Cluster Manager, distribuindo trabalho entre múltiplos Worker Nodes

🔗 Conceitos Fundamentais

RDDs (Resilient Distributed Datasets): Estruturas de dados fundamentais do Spark – coleções imutáveis de objetos particionados e tolerantes a falhas distribuídos pelo cluster

DAG (Directed Acyclic Graph): Grafo acíclico direcionado que representa a sequência de transformações aplicadas aos RDDs, permitindo otimização do plano de execução

⚙️ Fluxo de Execução

1. **Submissão:** Aplicação é submetida ao Driver Program
2. **Planejamento:** Código é traduzido em plano lógico e físico
3. **DAG:** Construção do grafo de estágios e tarefas
4. **Execução:** Tarefas distribuídas aos Executors

Componentes Principais e Modelo de Execução



Driver Program

Coordenador central da aplicação Spark

- Contém o SparkContext
- Traduz código em plano de execução
- Coordena com Cluster Manager
- Monitora execução do job



Cluster Manager

Gerencia recursos do cluster

Standalone Modo embutido

YARN Hadoop

Mesos Genérico

K8s Containers



Executors

Processos worker que executam tarefas

- Executam tarefas atribuídas
- Armazenam dados em cache
- Realizam operações shuffle
- Retornam resultados ao Driver

⚡ Otimizações de Performance

Lazy Evaluation

Não processa até necessário

Data Locality

Processa perto dos dados

In-Memory

Evita I/O em disco

Speculative

Duplica tarefas lentas

🔗 DAG Scheduler

Divide o DAG em estágios baseado em dependências:

1 Identifica wide dependencies (shuffle boundaries)

2 Cria estágios independentes

3 Task Scheduler distribui tarefas

RDDs: Resilient Distributed Datasets

Características Fundamentais

Imutabilidade

Uma vez criado, não pode ser modificado. Novos RDDs são criados via transformações

Lineage

Rastreamento de transformações permite recomputação automática em caso de falhas

Particionamento

Dados divididos em partições processadas em paralelo por diferentes nós

Lazy Evaluation

Transformações não são executadas imediatamente – apenas quando uma ação é chamada

</>Exemplo de Código

```
# Criar RDD
rdd = spark.sparkContext.parallelize([1, 2, 3, 4, 5])

# Transformações (lazy)
dobrado = rdd.map(lambda x: x * 2)
filtrado = dobrado.filter(lambda x: x > 4)

# Ação (executa)
resultado = filtrado.collect()

# Output: [6, 8, 10]
```

Tipos de Transformações

→ Narrow Transformations

Não requerem shuffle – dados de uma única partição

map()

filter()

flatMap()

union()

↔ Wide Transformations

Requerem shuffle – dados de múltiplas partições

groupByKey()

reduceByKey()

join()

distinct()

▶ Ações Comuns

collect()

Retorna todos elementos

count()

Conta elementos

reduce()

Agrega elementos

take(n)

Retorna n elementos

saveAsTextFile()

Salva em arquivo

The ETL process

ETAPA 02

Processamento de Dados (ETL)

Pipelines de ETL (Extract, Transform, Load), operações com DataFrames e otimização de processamento de dados em larga escala

Extract

Retrieves and verifies data from various sources

Transform

Processes and organizes extracted data so it is usable

Load

Moves transformed data to a data repository



Pipeline ETL com Apache Spark



Extract

Leitura de múltiplas fontes de dados

Sistemas de Arquivos

HDFS, S3, Azure Blob, GCS

Bancos de Dados

JDBC, PostgreSQL, MySQL, Oracle

Streaming

Kafka, Kinesis, Pulsar



Transform

Processamento e enriquecimento

Limpeza

Remoção de nulos, deduplicação

Agregação

groupBy, pivot, rollup

Enriquecimento

Joins, lookups, UDFs



Load

Escrita em destinos otimizados

Data Warehouses

Snowflake, Redshift, BigQuery

Data Lakes

Delta Lake, Iceberg, Hudi

NoSQL

Cassandra, MongoDB, HBase



Formatos de Arquivo Suportados

CSV

Texto plano

JSON

Semi-estruturado

Parquet

Colunar + compressão

ORC

Colunar Hadoop

Avro

Binário + schema

Delta

ACID + versionamento

Parquet é o formato recomendado devido à compressão eficiente, predicate pushdown e suporte nativo ao Spark

</>Exemplo de Pipeline ETL

```
# Extract
df = spark.read.format("parquet") \
.load("s3://bucket/raw/")

# Transform
df_clean = df.filter("amount > 0") \
.groupBy("category") \
.agg(sum("amount").alias("total"))

# Load
df_clean.write.format("delta") \
.mode("overwrite") \
.save("/mnt/processed/")
```


Otimizações e Boas Práticas em ETL

🔽 Predicate Pushdown

Técnica que emprega filtros diretamente na fonte de dados antes de carregar no Spark, minimizando I/O

```
# Filtro aplicado na fonte (otimizado)
df = spark.read.parquet("path") \
    .filter("date >= '2024-01-01'")
```

✅ Parquet e ORC suportam pushdown nativo

📦 Particionamento e Bucketing

Particionamento

Distribui dados por colunas para paralelismo eficiente

```
df.write.partitionBy("year", "month")
```

Bucketing

Agrupar dados em buckets baseado em hash para joins eficientes

```
df.write.bucketBy(100, "user_id")
```

🔄 Redução de Shuffle

Operações que minimizam movimentação de dados entre nós

reduceByKey() vs groupByKey()

reduceByKey faz pré-agregação local antes do shuffle – mais eficiente

aggregateByKey()

Permite agregações complexas com zero value

🗄️ Cache e Persistência

Armazena DataFrames intermediários em memória/disco para reutilização

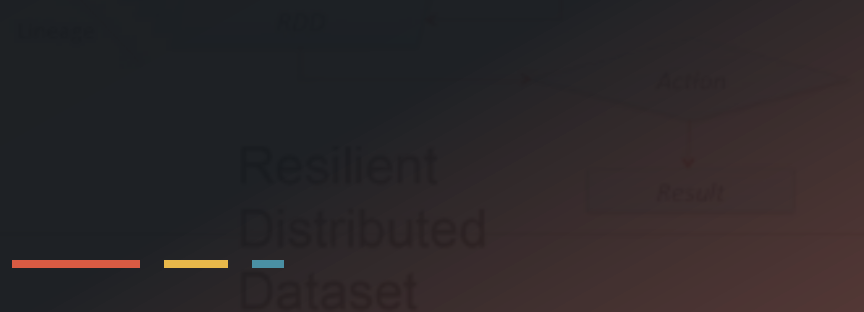
cache()	MEMORY_ONLY
persist(MEMORY_AND_DISK)	Memória + disco
persist(DISK_ONLY)	Apenas disco

Dica: Use `broadcast()` para DataFrames pequenos (< 10MB) em joins – evita shuffle completo

ETAPA 03

Spark SQL e DataFrames

Processamento estruturado de dados usando Spark SQL, DataFrames, Datasets e o Catalyst Optimizer



←
Interpreter &
Optimizer



DataFrames e Spark SQL

🗄️ DataFrames vs RDDs

Otimização Automática

Catalyst Optimizer cria plano de execução otimizado automaticamente

Schema Inference

Detecção automática de tipos de dados e validação de schema

Integração SQL

Consultas SQL diretamente sobre DataFrames com Spark SQL

Consistência Cross-Language

Mesma API e comportamento em Python, Scala, Java e R

📦 Datasets (Scala/Java)

Extensão tipada dos DataFrames com verificação em compile-time

```
// Scala - Dataset tipado
case class Pessoa(nome: String, idade: Int)
val ds: Dataset[Pessoa] = spark
  .read
  .json("pessoas.json")
  .as[Pessoa]
```

🔗 Operações com DataFrames

```
# Criar DataFrame
df = spark.read.json("dados.json")

# Select e Filter
df.select("nome", "idade") \
  .filter(df.idade > 18)

# GroupBy e Agregação
df.groupBy("categoria") \
  .agg(avg("preco").alias("media"))

# Join
df1.join(df2, "id", "inner")
```

🔄 Spark SQL

```
# Registrar como view temporária
df.createOrReplaceTempView("pessoas")

# Executar consulta SQL
resultado = spark.sql("""
SELECT categoria, AVG(preco) as media
FROM pessoas
WHERE idade > 18
GROUP BY categoria
""")
```

100x

Mais rápido que MapReduce

4

Linguagens suportadas

SQL

Integração nativa

Catalyst Optimizer e Tungsten Engine

Catalyst Optimizer

Otimizador de consultas baseado em regras e custos que transforma planos lógicos em planos físicos otimizados

1 **Unresolved Logical Plan**
Plano inicial sem validação de schema

2 **Analysis & Resolution**
Validação de tabelas, colunas e tipos

3 **Logical Optimization**
Aplicação de regras de otimização

4 **Physical Planning**
Geração de múltiplos planos físicos

5 **Cost Model Selection**
Seleção do plano de menor custo

Regras de Otimização

Constant Folding
Simplifica expressões com valores constantes

Predicate Pushdown
Empurra filtros para a fonte de dados

Projection Pruning
Seleciona apenas colunas necessárias

Join Reordering
Reordena joins para menor custo

Tungsten Engine

Projeto de otimização de CPU e memória

✓ Whole-Stage Code Generation

✓ Off-Heap Memory Management

✓ Cache-friendly Binary Format

Visualizar Plano de Execução

```
df.explain(True) # Mostra todos os planos: parsed, analyzed, optimized e physical
```

ETAPA 04

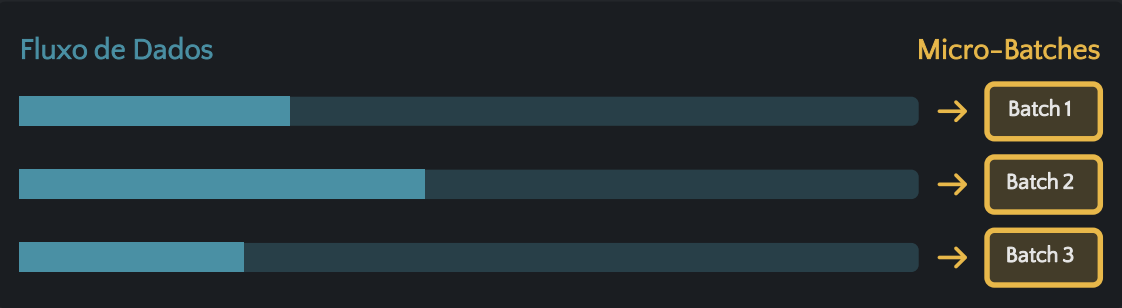
Streaming e Processamento em Tempo Real

Processamento de fluxo contínuo de dados usando Structured Streaming, incluindo conceitos de event time, watermarks e output modes

Structured Streaming: Arquitetura e Conceitos

Micro-Batch Processing

Modelo de processamento onde fluxos contínuos são tratados como uma série de micro-lotes, simplificando o modelo de programação



Event Time vs Processing Time

Event Time

Momento em que o evento ocorreu na fonte (timestamp do dado)

Processing Time

Momento em que o evento é processado pelo Spark (timestamp do sistema)

Output Modes

Append

Apenas novos registros

Adiciona apenas novos registros ao sink. Requer watermark para agregações

Update

Atualizações

Atualiza registros modificados no sink. Útil para agregações

Complete

Tabela completa

Reescreve toda a tabela a cada trigger. Para agregações totais

Watermarks

Limite de tempo para aceitar eventos atrasados, permitindo limpeza de estado

```
df.withWatermark("timestamp", "10 minutes")
```



Fontes e Sinks Suportados

- Kafka
- Kinesis
- Socket
- File
- Delta Lake

Modo em Tempo Real e Casos de Uso

⚡ Real-Time Mode (Spark 4.0)

Novo modo de processamento contínuo com latência na casa dos **milissegundos**, processando eventos à medida que chegam

- Latência P99**
~15-300ms
Dependendo da complexidade
- Streaming Shuffle**
Dados passam entre tarefas em memória, reduzindo overhead de coordenação
- Ativação Simples**
Uma única configuração - sem reescrita de código

</>Configuração

```
# Ativar modo em tempo real
spark.conf.set(
  "spark.sql.streaming.realTimeMode.enabled",
  "true"
)

# Usar trigger de tempo real
df.writeStream \
  .trigger(RealTimeTrigger.apply("5m")) \
  .start()
```

📊 Comparação: Micro-Batch vs Real-Time

Micro-Batch	Tradicional
⌚ Latência: segundos a minutos	
📅 Agendamento fixo	
💰 Mais eficiente em custo	
Real-Time Mode	NOVO
⚡ Latência: milissegundos	
∞ Processamento contínuo	
🔧 Para casos críticos	

🏠 Casos de Uso Reais

- Detecção de Fraude**
Banco processa transações em 200ms
- Personalização em Tempo Real**
E-commerce atualiza ofertas instantaneamente
- Feature Serving para ML**
App de delivery atualiza ETAs em milissegundos

ETAPA 05

Machine Learning com MLlib

Uso do MLlib para machine learning em escala, incluindo pipelines, transformers, estimators e avaliação de modelos



MLlib: Algoritmos e Componentes



Classificação

- Logistic Regression
- Naive Bayes
- Decision Trees
- Random Forests
- GBT (Gradient Boosting)
- Linear SVM
- Factorization Machines



Regressão

- Linear Regression
- Generalized Linear
- Survival Regression
- Decision Trees
- Random Forests
- GBT Regressor
- Isotonic Regression



Clustering

- K-Means
- Gaussian Mixture
- Bisecting K-Means
- LDA (Topic Modeling)
- Power Iteration
- Recomendação**
ALS (Alternating Least Squares)



Feature Engineering

- | | |
|----------------------------|-------------------------------|
| Tokenizer / RegexTokenizer | StopWordsRemover |
| HashingTF / IDF | Word2Vec |
| StringIndexer | OneHotEncoder |
| VectorAssembler | StandardScaler / MinMaxScaler |



Utilitários

- Álgebra Linear Distribuída**
SVD, PCA, matrizes distribuídas
- Estatísticas**
Correlação, testes de hipótese, resumo
- Avaliação de Modelos**
Métricas de classificação, regressão, clustering

100x

Mais rápido que MapReduce

4

Linguagens (Python, Scala, Java, R)

RDD → DF

API baseada em DataFrames

GPU

Suporte a aceleração

Pipelines de ML: Transformers e Estimators

↔ Arquitetura de Pipelines

T Transformer

Aplica transformação aos dados. Implementa método **transform()**

StringIndexer, OneHotEncoder

VectorAssembler, StandardScaler

E Estimator

Treina um modelo nos dados. Implementa método **fit()**

LogisticRegression, RandomForest

KMeans, ALS

M Model

Resultado do fit() de um Estimator - é um Transformer

🔗 Pipeline Completo

```
# Definir estágios
indexer = StringIndexer(
    inputCol="category",
    outputCol="categoryIndex")
assembler = VectorAssembler(
    inputCols=["categoryIndex", "value"],
    outputCol="features")
lr = LogisticRegression(
    featuresCol="features",
    labelCol="label")
# Criar pipeline
pipeline = Pipeline(stages=[
    indexer, assembler, lr])
# Treinar
model = pipeline.fit(train_df)
# Prever
predictions = model.transform(test_df)
```

⚙️ Tuning de Hiperparâmetros

```
param_grid = ParamGridBuilder() \
    .addGrid(lr.regParam, [0.1, 0.01]) \
    .addGrid(lr.maxIter, [10, 20]) \
    .build()
cv = CrossValidator(
    estimator=pipeline,
    estimatorParamMaps=param_grid,
    evaluator=BinaryClassificationEvaluator(),
    numFolds=5)
```

Data Mining e Análise de Dados com Spark



Padrões Frequentes

FP-Growth

Algoritmo escalável para mineração de itemsets frequentes

Association Rules

Descoberta de regras de associação (market basket analysis)

PrefixSpan

Mineração de padrões sequenciais



Detecção de Anomalias

K-Means + Distância

Pontos com maior distância ao centróide são anomalias

Isolation Forest

Algoritmo baseado em árvores para detecção de outliers

LOF (Local Outlier Factor)

Detecta anomalias baseado na densidade local



Processamento de Texto

Word2Vec

Embeddings de palavras em vetores densos

TF-IDF

Importância de termos em documentos

CountVectorizer

Vetorização baseada em frequência



Análise de Grafos (GraphX)

PageRank

Importância de nós em grafos

Connected Components

Identificação de componentes conectados

Triangle Count

Contagem de triângulos em grafos

Shortest Paths

Caminhos mais curtos entre nós



Integração com Ecossistema

Pandas UDFs

Aplica funções pandas em DataFrames Spark

TensorFlow / PyTorch

Treinamento distribuído via Horovod

XGBoost

Integração nativa no Databricks Runtime ML

Scikit-learn

Via pandas UDFs para datasets pequenos



Casos de Uso de Data Mining

Segmentação de clientes

Recomendação de produtos

Detecção de fraude

Análise de sentimento

Churn prediction

ETAPA 06

Tecnologias Emergentes e Boas Práticas

Tecnologias emergentes como Delta Lake, Spark 3.x/4.x, Adaptive Query Execution, e melhores práticas para produção



Delta Lake: Transações ACID no Data Lake

O que é Delta Lake?

Camada de storage open source que traz **transações ACID**, versionamento e qualidade de dados aos data lakes baseados em Parquet

- Atomicidade**
Operações são atômicas – tudo ou nada
- Consistência**
Dados sempre consistentes após operações
- Isolamento**
Transações concorrentes não interferem
- Durabilidade**
Dados persistidos após commit

★ Recursos Principais

- Time Travel**
Acesso a versões históricas dos dados via timestamp ou versão
- Schema Enforcement & Evolution**
Validação de schema e evolução controlada
- Upserts e Deletes**
Operações MERGE, UPDATE, DELETE em data lakes
- OPTIMIZE**
Compactação automática de arquivos pequenos
- Z-Ordering**
Colocalização de informações relacionadas para leituras eficientes
- Change Data Feed (CDF)**
Rastreamento de mudanças em nível de linha

</> Exemplo de Uso

```
# Time Travel
df = spark.read \
    .format("delta") \
    .option("versionAsOf", 0) \
    .load("/path")

# OPTIMIZE
OPTIMIZE delta_table
ZORDER BY (column)
```



Spark 3.x/4.0: Novidades e Adaptive Query Execution

Adaptive Query Execution (AQE)

Otimização **runtime** que re-otimiza consultas durante execução baseado em estatísticas coletadas

Dynamic Join Strategy

Converte sort-merge joins para broadcast hash joins se dados forem pequenos

Dynamic Coalescing

Reduz número de partições após shuffle se muitas partições pequenas

Dynamic Skew Handling

Detecta e mitiga skew de dados em joins e agregações

Empty Relation Propagation

Otimiza queries podando partes do plano com resultados vazios

Novidades Spark 4.0

Real-Time Mode

Latência de milissegundos para streaming

transformWithState

Stateful streaming com controle de estado

Tipo VARIANT

Suporte nativo a dados semi-estruturados

State Reader API

Leitura de estado de streaming para debug

Photon Engine (Databricks)

Motor vetorizado escrito em C++ para aceleração de queries

✓ Até 18x mais rápido em star schema

✓ 5x mais rápido em scans

✓ Cache otimizado em NVMe SSD



Configuração do AQE

```
spark.sql.adaptive.enabled = true # Habilita AQE (padrão em Spark 3.2+)
```

Ecosistema e Tecnologias Complementares



Ingestão

Apache Kafka

Streaming de eventos em tempo real

Apache Pulsar

Mensageria cloud-native

AWS Kinesis

Streaming na AWS



Orquestração

Apache Airflow

Orquestração de workflows

Databricks Workflows

Jobs e pipelines nativos

Prefect / Dagster

Orquestração moderna



MLOps

MLflow

Gerenciamento de ciclo de ML

Feature Store

Gerenciamento de features

AutoML

Automação de modelos



Qualidade de Dados

Great Expectations

Validação e documentação de dados

Apache Griffin

Framework de qualidade de dados

Soda Core

Verificação de qualidade em pipelines



Deployment

Kubernetes

Orquestração de containers

Docker

Containerização de aplicações

Cloud Providers

AWS EMR, Azure HDInsight, GCP Dataproc



Tendências Emergentes

GPU Acceleration

AutoML

Real-time ML

Federated Learning

Lakehouse Architecture

Casos de Uso e Aplicações Reais

Netflix

Escala

Processa **10+ PB diários** de dados de streaming

Casos de Uso

- Sistema de recomendação personalizado
- Análise de comportamento de usuários
- Otimização de CDN e streaming

Uber

Pipelines em Tempo Real

Métricas de rede global atualizadas continuamente

Tecnologias

Spark Streaming + Hudi para upserts incrementais

Serviços Financeiros

Detecção de Fraude

Processa transações em **200ms** com Structured Streaming

Autorização de Pagamentos

Network International: P99 de **15ms** com Real-Time Mode

E-commerce e Mídia

Personalização em Tempo Real

Recomendações atualizadas instantaneamente durante navegação

Streaming OTT

Recomendações de conteúdo pós-visualização

10+ PB

Processados diariamente (Netflix)

200ms

Detecção de fraude

15ms

Autorização P99

1 PB+

Streaming/mês (Databricks)

Conclusão da Aula e Próximos Passos

O **Apache Spark** consolidou-se como o **framework líder** para processamento distribuído de dados, oferecendo uma **plataforma unificada** para ETL, análise, ML e streaming.

✓ Inovações Contínuas

Com as inovações do **Spark 4.0**, **Delta Lake** e tecnologias emergentes, o ecossistema continua evoluindo para atender demandas de **latência cada vez menores** e **volumes cada vez maiores**

🎓 Domínio Completo

Para maximizar resultados, é essencial dominar não apenas os **conceitos técnicos**, mas também as **melhores práticas** de arquitetura, otimização e governança de dados

🏠 Próximos Passos Recomendados

1. Prática Hands-on

Crie pipelines reais em Databricks Community Edition

2. Certificações

Databricks Data Engineer Associate/Professional

3. Comunidade

Participe de meetups e contribua com projetos open source